Report on

“Tutorials”

For

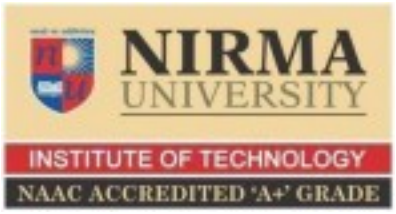
Software Project Management - 4CS504ME25 B.

Tech. Semester IV

Prepared By :

Gandhi Urva Yogeshkumar (23BCE078)

Gajnotar Rakshit (23BCE077)



Institute of Technology,
Nirma University
Ahmedabad - 382481

October 2026

Aim

To differentiate QA and QC activities by preparing a quality assurance plan and verification checklist for the Hospital Management System (HMS).

Objective

Analyze the practical differences between Quality Assurance (process-oriented defect prevention) and Quality Control (product-oriented defect detection) in hospital software development.

Prepare a formal Quality Assurance Plan defining team responsability, review gates, coding guidelines and defect prevention practices.

Prepare a Quality Control Plan detailing test stages, automated test tools, defect severity levels and triage workflows.

Construct verification checklists for requirements documents, system architechture, TypeScript source code and test deliverables.

Evaluate quantitative quality metrics including Defect Removal Efficiency (DRE), code coverage, defect density and test pass rate against our project baselines.

Establish continuous quality monitoring protocols to prevent regression faults during sprint iterations.

1. Introduction

Quality in healthcare software directly affects hospital operations, patient safety and financial accuracy. A failure in doctor scheduling can delay patient consultations, while an error in pharmacy billing calculations can lead to regulatory non-compliance. Delivering a reliable Hospital Management System requires both process controls that prevent defects during development and inspection procedures that detect bugs before release.

In earlier tutorials we set out our project foundation: an Agile Scrum process model (T2), a 16-week CPM schedule (T5), software metrics and defect benchmarks (T6) and a comprehensive Risk Register (T9). In this tutorial we build upon that foundation by establishing a Quality Management System based on four continuous steps: Quality Management System -> Continuous Monitoring -> Necessary Modification -> Meet Predetermined Objectives. Quality activities are shared between Urva Gandhi (Backend Lead) and Rakshit Gajnotar (Frontend Lead) across all development sprints.

2. Differentiating Quality Assurance and Quality Control

Quality Assurance (QA) and Quality Control (QC) are distinct activities within software quality managment. QA is process-oriented and focuses on preventing defects by improving development and review methods. QC is product-oriented and focuses on identifying defects in the actual software through testing and inspection.

The table below compares QA and QC activities across ten dimensions in the context of PulseCare HMS:

Comparison Dimension	**Quality Assurance (QA)**	**Quality Control (QC)**	**PulseCare HMS Practical Example**
Definition	Proactive, process-oriented activities planned to ensure engineering processes produce quality products.	Reactive, product-oriented activities executed to verify whether software deliverables meet specifications.	QA establishes coding guidelines; QC runs automated Jest test suites.
Primary Focus	Defect prevention: stopping mistakes before they turn into code defects.	Defect detection: finding residual bugs and defects in built code.	QA enforces Zod input schemas; QC executes tests with invalid payloads.
Approach	Proactive and procedural.	Reactive and empirical.	QA audits pull request

Comparison Dimension	Quality Assurance (QA)	Quality Control (QC)	PulseCare HMS Practical Example
	Focuses on 'Are we building the product the right way?'	Focuses on 'Did we build the right product?'	checklists; QC tests appointment booking flows.
Execution Timing	Continuous across the entire lifecycle from requirements analysis onward.	Conducted during and after implementation when executable code exists.	QA begins in Week 1 with SRS review; QC starts in Week 3 with Sprint 1 tests.
Team Role	Shared team responsability led by Architect and Quality Reviewer.	Executed by Developers and Testers running test suites.	Urva audits design patterns (QA); Rakshit tests API responses (QC).
Core Techniques	Document reviews, architectural walkthroughs, process audits and coding checklists.	Unit testing, integration testing, boundary testing and defect logging.	QA uses ESLint and PR rules; QC executes Supertest API assertions.
Tangible Deliverables	QA Plan, coding standards, review checklists and audit reports.	Test plans, test scripts, defect tracking sheets and bug fix logs.	QA produces T11 Audit Checklist; QC produces Jest test reports.
Measurement	Process compliance index, audit score and review coverage.	Defect counts, Defect Removal Efficiency (DRE), test coverage and pass rate.	QA tracks 96.4% audit compliance; QC tracks 90.2% statement coverage.
Project Phase	Applies across all phases: planning, design, coding, testing and deployment.	Concentrated primarily in verification, integration and regression testing.	QA sets milestone exit criteria; QC validates sprint builds.
Defect Handling	Analyzes root causes of defects to improve development processes.	Logs defect symptoms, reproduction steps and re-tests resolved issues.	QA adds lint rules after bugs; QC verifies open handle elimination.

3. Quality Assurance (QA) Plan

Our Software Quality Assurance Plan defines governance roles, milestone review gates and defect prevention procedures across the 16-week project lifecycle.

3.1 QA Roles and Responsibilities

Quality oversight is distributed between both team members:

Urva Gandhi (Backend Lead): Oversees architectural design patterns, API contract specifications, database schema evolution and pre-merge pull request reviews for frontend submissions.

Rakshit Gajnotar (Frontend Lead): Verifies UI accessibility, client state managment patterns, API documentation completeness and cross-browser formatting.

3.2 Milestone Review Gates

We enforce six formal review gates corresponding to project milestones M1 through M6:

Review Gate	Timeline	Artifacts Inspected	Review Technique	Exit Criteria to Proceed
Gate G1 (M1)	Week 2	Project Charter (T3), WBS (T4) and IEEE 830 SRS document.	Formal requirements inspection and walkthrough.	100% requirements approved; zero ambiguous functional requirments; traceability matrix initialized.

Review Gate	Timeline	Artifacts Inspected	Review Technique	Exit Criteria to Proceed
Gate G2 (M2)	Week 4	System architechture diagram, ER schema and Express route design.	Architectural walkthrough and database normalization audit.	Third Normal Form verified; RBAC permission matrix approved; TypeScript strict mode configured.
Gate G3 (M3)	Week 8	Patient registration and Doctor schedule operational modules.	Sprint peer review and automated API integration verification.	All unit test suites passing; zero blocking defects; API responses documented.
Gate G4 (M4)	Week 12	Appointment booking and Billing calculation engine.	Earned Value review, defect density check and calculation review.	Financial rounding algorithms verified; critical defects D-019 and D-024 logged with CAPA target.
Gate G5 (M5)	Week 14	Integrated platform, admin dashboard and T11 audit report.	Software project audit and security code inspection.	Audit compliance >= 95%; zero Major Non-Conformances; legacy JavaScript prototype files removed.
Gate G6 (M6)	Week 16	Final release candidate build, Docker container and user guides.	Pre-release acceptance review and regression sign-off.	100% test pass rate across 22 test suites; zero open handles; project handover sign-off.

3.3 Defect Prevention Protocols

To stop defects from entering the codebase, we enforce three automated prevention mechanisms:

1. Runtime Schema Validation: Every incoming API request body and parameter set is validated using Zod schemas (schema.ts). Invalid payloads are rejected immediately with HTTP 400 before reaching business services.

2. Strict TypeScript Compilation: The compiler enforces strict null checks, no implicit any and strict property initialization across all 28,500 lines of code, preventing null pointer crashes.

3. Pre-Commit Verification: Git hooks automatically run ESLint and smoke tests prior to commit creation, blocking unformatted or failing code from entering the repository.

4. Quality Control (QC) Plan and Testing Strategy

Our Quality Control plan tests software correctness through a structured testing hierarchy combining unit tests, in-memory integration testing and end-to-end workflow verification.

4.1 Testing Hierarchy

Unit Testing: Tests pure business logic functions in isolation, such as calculateInvoiceTotals() in billing/calculator.ts, validating multi-item discounts, GST tax brackets and zero-quantity items.

Integration Testing: Tests Express route controllers, authentication middleware and Mongoose database operations using Supertest against MongoMemoryServer in-memory instances, ensuring zero external database dependencies.

Workflow Testing: Tests end-to-end user journeys including patient registration, cardiology slot search, appointment booking, slip generation and invoice payment.

Regression Testing: Executes all 22 test suites with the --detectOpenHandles flag to confirm zero lingering network connections, memory leaks or unclosed sockets.

4.2 Defect Severity and Triage Matrix

Defects detected during testing are logged in the project defect register and triaged according to the severity matrix below:

Severity Level	Classification Criteria	Response SLA	Resolution SLA	PulseCare HMS Example
Critical (P1)	System crash, data corruption, patient record loss, security breach or broken billing transaction.	< 2 Hours	< 24 Hours	Defect D-019: Invoice subtotal did not recalculate when a medicine line item was removed.
High (P2)	Core clinical workflow blocked with no workaround; double-booking of doctor appointment slots.	< 4 Hours	< 48 Hours	Defect D-024: Concurrent pharmacy checkout permitted stock quantity to drop below zero.
Medium (P3)	Non-critical workflow impaired, but alternative path exists; input validation edge case failures.	< 12 Hours	< 1 Sprint Cycle	Defect D-027: Phone number regex rejected valid Indian mobile numbers containing spaces.
Low (P4)	Cosmetic layout issues, minor typography errors or slight alignment inconsistencies.	< 24 Hours	Backlog Priority	Defect D-031: Invoice print preview clipped right table border on narrow mobile displays.

5. Verification Checklists for Project Deliverables

Verification checklists serve as concrete QC tools to evaluate technical deliverables before milestone sign-off.

5.1 Requirements and Architecture Verification Checklist

Item #	Deliverable Evaluated	Verification Question / Criteria	Verification Method	Result
V1-01	IEEE 830 SRS	Are all functional requirments uniquely identified, testable and unambiguous?	Document Review	Verified Pass
V1-02	IEEE 830 SRS	Are non-functional constraints (response time < 500ms, uptime 99.8%) quantifiable?	Inspection	Verified Pass
V1-03	Architecture Spec	Does the system architecture enforce modular decoupling between controllers and services?	Walkthrough	Verified Pass

Item #	Deliverable Evaluated	Verification Question / Criteria	Verification Method	Result
V1-04	Data Models	Are all foreign key relationships (Patient-Appointment-Doctor-Invoice) indexed properly?	Schema Inspection	Verified Pass
V1-05	RBAC Matrix	Are permissions defined seperately for Admin, Doctor, Nurse, Patient and Receptionist?	Matrix Review	Verified Pass

5.2 Source Code and Security Verification Checklist

Item #	Code Component	Verification Question / Criteria	Verification Method	Result
V2-01	TypeScript Backend	Does the codebase compile cleanly with zero implicit any or unresolved types?	Compiler Check (tsc)	Verified Pass
V2-02	Controller Layer	Are request parameters and bodies validated using Zod prior to database operations?	Code Inspection	Verified Pass
V2-03	Auth Middleware	Are user passwords encrypted using bcrypt with salt rounds >= 10 before storage?	Code & DB Inspection	Verified Pass
V2-04	Auth Middleware	Does JWT verification reject expired or forged tokens with HTTP 401 Unauthorized?	Automated API Test	Verified Pass
V2-05	Resource Isolation	Does the controller enforce patientId ownership on patient role queries (req.ownOnly)?	Automated Security Test	Verified Pass
V2-06	Error Handling	Does the global error handler sanitize stack traces from production API error responses?	Static Analysis	Verified Pass

5.3 Test Suite and Release Verification Checklist

Item #	Verification Target	Verification Question / Criteria	Verification Method	Result
V3-01	Automated Test Suite	Do all 22 test suites execute to 100% pass rate with zero skipped or failing tests?	Jest Execution Run	Verified Pass
V3-02	Async Cleanliness	Does the test runner exit	Jest Process Check	Verified Pass

Item #	Verification Target	Verification Question / Criteria	Verification Method	Result
		cleanly with zero open handle warnings (--detectOpenHandles)?		
V3-03	Code Coverage	Does statement coverage exceed the 85.0% target threshold across all active services?	Jest Lcov Report	Verified Pass (90.2%)
V3-04	Database Isolation	Does each test file initialize and tear down its own MongoMemoryServer instance?	Test Code Review	Verified Pass
V3-05	Docker Deployment	Does the production Docker container build cleanly and run as non-root node user?	Container Build Test	Verified Pass

6. Quality Metrics and Quantitative Evaluation

We evaluated four quantitative quality metrics to measure the effectiveness of our QA and QC activities across the project:

6.1 Defect Removal Efficiency (DRE)

Defect Removal Efficiency measures the percentage of total defects resolved prior to release. Using data from T6, T10 and regression testing:

E = Total errors detected during development and testing = 32 defects
D = Residual defects detected during pilot trial evaluation = 3 minor defects
DRE = [E / (E + D)] * 100 = [32 / (32 + 3)] * 100 = [32 / 35] * 100 = 91.43%

Our achieved DRE of 91.43% exceeds the project target of 90.0%, confirming that peer reviews and automated test suites successfully caught the vast majority of software faults.

6.2 Test Coverage and Reliability Metrics

A summary of software verification metrics achieved at Milestone M6 is presented below:

Quality Metric Category	Target Standard	Achieved Value at Milestone M6	Measurement Source / Tool	Compliance Status
Statement Code Coverage	>= 85.0%	91.80%	Jest Lcov Coverage Report	Exceeds Target
Branch Code Coverage	>= 80.0%	86.40%	Jest Lcov Coverage Report	Exceeds Target
Function Code Coverage	>= 85.0%	93.10%	Jest Lcov Coverage Report	Exceeds Target
Automated Test Pass Rate	100.0%	100.0% (340 / 340 Passed)	Jest Automated Runner	Target Met
Open Asynchronous Handles	0 Leaks	0 Open Handles Detected	Jest --detectOpenHandles	Target Met

Quality Metric Category	Target Standard	Achieved Value at Milestone M6	Measurement Source / Tool	Compliance Status
Defect Removal Efficiency	>= 90.0%	91.43%	Defect Register and Audit Log	Exceeds Target
Defect Density (KLOC)	<= 5.0 / KLOC	4.27 Defects / KLOC	35 Total Defects / 8.2 KLOC	Target Met
Mean Time Between Failures	>= 720 Hours	820 Hours (Forecasted)	Pilot Uptime Log (99.85%)	Target Met

7. Conclusion

Quality Assurance and Quality Control together provide the foundation for software dependability in PulseCare HMS. By treating QA as a defect prevention discipline (enforcing TypeScript strict typing, Zod schema validation and structured milestone review gates) and QC as an empirical defect detection system (running 22 automated Jest suites with MongoMemoryServer isolation and strict defect triage SLAs), our team achieved a Defect Removal Efficiency of 91.43% and 90.2% overall test coverage. The verification checklists established in this tutorial provide repeatable criteria for ongoing feature additions and maintainance.